



PELO FUTURO DO TRABALHO

Objetivo

- Introdução a conexão MySQL +Java

Professor



- Professor nas áreas de tecnologia, administração e educação.
- Experiência docente no SENAI, Instituto Federal Catarinense e Uniasselvi.
- Mestrando em Educação (Instituto Federal Catarinense)
 - Especialização em Educação a Distância na Educação Profissional e Tecnológica (IFC)
 - Especialização em Abordagens Ativas e suas metodologias (IFSC)
 - MBA em Gestão de Projetos (Uniasselvi)
 - MBA em Gestão de Tecnologia da Informação (IGTI)
 - MBA em Gestão Estratégica de Pessoas (Estácio)
- Graduação em Sistemas de Informação
- Licenciatura em Matemática
- Pesquisa voltada para Inteligência Artificial e inclusão.
- Atua com pesquisa, educação profissional, tecnologias educacionais e inovação na educação.



Suggestions for Approval

- Be present body and soul
- Do not leave the room during classes
- Take notes of what is explained
- After each class, read the material in the recommended bibliography or textbook
- Clarify your doubts with the teacher
- Don't let the content pile up
- Don't leave work to the last minute
- Do the work for the class, not for yourself
- Work as a team
- Follow the scientific method



Características da aula

- Previstas: 3 avaliações
 - Conceitos apresentados (slides, apresentação verbal e demonstração de exemplos) em sala.

Conexões de banco

- Conexão Direta: Mais simples e usada em aulas iniciais.
- Conexão com DAO: Tela → DAO → JDBC → Banco
- Connection Factory: Criar uma classe responsável somente pela conexão.
- Pool de Conexões: o sistema mantém conexões prontas. Aplicação → Pool → Banco
- Conexão via ORM: JPA Hibernate. O Java vira objeto e o framework gera SQL automaticamente.
- Conexão Cliente-Servidor: Cliente → Internet/Rede → Servidor MySQL
- Conexão em Nuvem

JDBC e ORM

JDBC é como dirigir manualmente.
ORM é como dirigir carro automático.

JDBC vs ORM (Hibernate)

Duas formas de acessar o banco de dados em aplicações Java

JDBC (Java Database Connectivity)	VS	ORM (Object Relational Mapping) Ex.: Hibernate / JPA
 Aplicação Java → Driver JDBC → Banco de Dados (MySQL)	 IDEIA PRINCIPAL	 Aplicação Java (objetos) → ORM (Hibernate/JPA) → Banco de Dados (MySQL)
O desenvolvedor escreve SQL e o JDBC envia os comandos diretamente para o banco.		O desenvolvedor trabalha com objetos Java e o ORM converte automaticamente para SQL.
• SQL escrito manualmente • Mais controle sobre o SQL • Mais código repetitivo	 ABORDAGEM	• Trabalha com entidades (classes Java) • SQL gerado automaticamente • Menos código repetitivo
<pre>String sql = "INSERT INTO aluno (nome, idade, curso) VALUES (?, ?, ?)"; PreparedStatement ps = conexao.prepareStatement(sql); ps.setString(1, "João Silva"); ps.setInt(2, 20); ps.setString(3, "SI"); ps.executeUpdate();</pre>	 EXEMPLO - INSERIR DADO	<pre>Aluno aluno = new Aluno(); aluno.setNome("João Silva"); aluno.setIdade(20); aluno.setCurso("SI"); entityManager.persist(aluno); // Hibernate/JPA</pre>
<pre>String sql = "SELECT id, nome, idade, curso FROM aluno WHERE id = ?"; PreparedStatement ps = conexao.prepareStatement(sql); ps.setInt(1, 1); ResultSet rs = ps.executeQuery(); if (rs.next()) { String nome = rs.getString("nome"); int idade = rs.getInt("idade"); }</pre>	 EXEMPLO - CONSULTAR DADO	<pre>Aluno aluno = entityManager.find(Aluno.class, 1); // ou com JPQL List<Aluno> lista = entityManager. createQuery("SELECT a FROM Aluno a", Aluno.class) .getResultList();</pre>
 • Total controle sobre o SQL gerado • Melhor performance em consultas complexas • Sem dependência de frameworks	VANTAGENS	• Menos código (mais produtividade) • Mais segurança (menos SQL manual) • Portabilidade entre bancos • Mapeamento objeto-relacional
 • Mais código repetitivo • Maior propensão a erros (SQL manual) • Manutenção mais trabalhosa	DESVANTAGENS	• Curva de aprendizado • Pode gerar SQL menos otimizado em alguns casos • Dependência de frameworks/bibliotecas
 Sistemas simples, consultas muito específicas, relatórios complexos, máximo desempenho.	INDICAÇÃO DE USO	Sistemas médios e grandes, CRUDs, produtividade, manutenção e evolução rápida.



RESUMO

JDBC = você escreve o SQL

Mais trabalho manual, mais controle.



ORM = você trabalha com objetos

Menos código, mais produtividade.

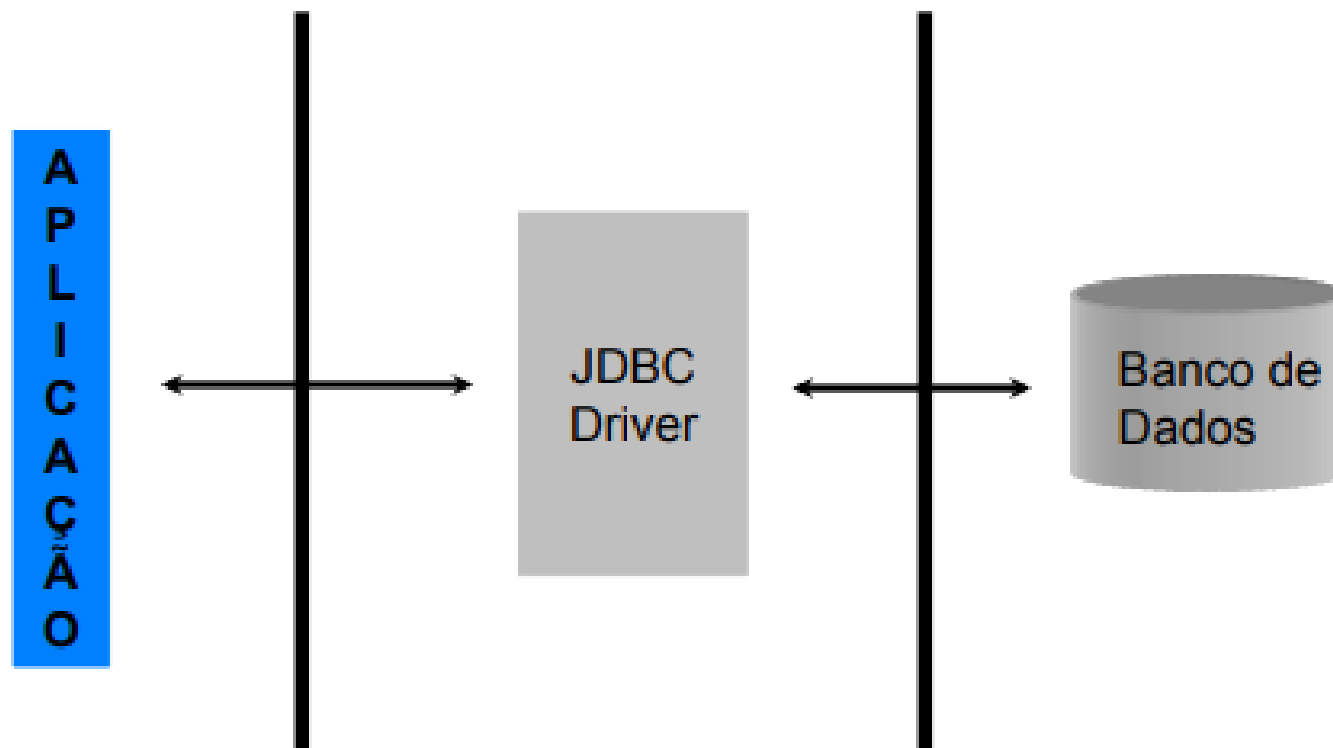


IMPORTANTE

ORM não substitui o JDBC.
ORM usa JDBC por baixo dos panos para se comunicar com o banco!

Utilizando SQL em Java

JDBC (Java Database Connectivity) é um conjunto de classes e interfaces escritas em Java que fazem o envio de instruções SQL para qualquer banco de dados relacional. Para cada banco de dados, há um driver JDBC.



Pacote java.sql

- O pacote java.sql permite conectar aplicações Java a bancos de dados relacionais.
- Pode ser utilizado em aplicações Web e Desktop.
- Cada banco de dados possui seu próprio driver de conexão.
- O driver faz a comunicação entre o Java e o banco de dados.
- Para usar o MySQL em Java, é necessário baixar o driver do MySQL.
- Esse driver permite que a aplicação Java se conecte ao banco MySQL.

Pacotes de Conexão

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;
```

Criando uma Conexão

- JDBC referencia bancos de dados individualmente através do formato da URL
 - `jdbc : <subprotocol> : <subname>`
 - subprotocol: nome do driver
 - subname: nome e caminho da base de dados
 - Exemplos de url:

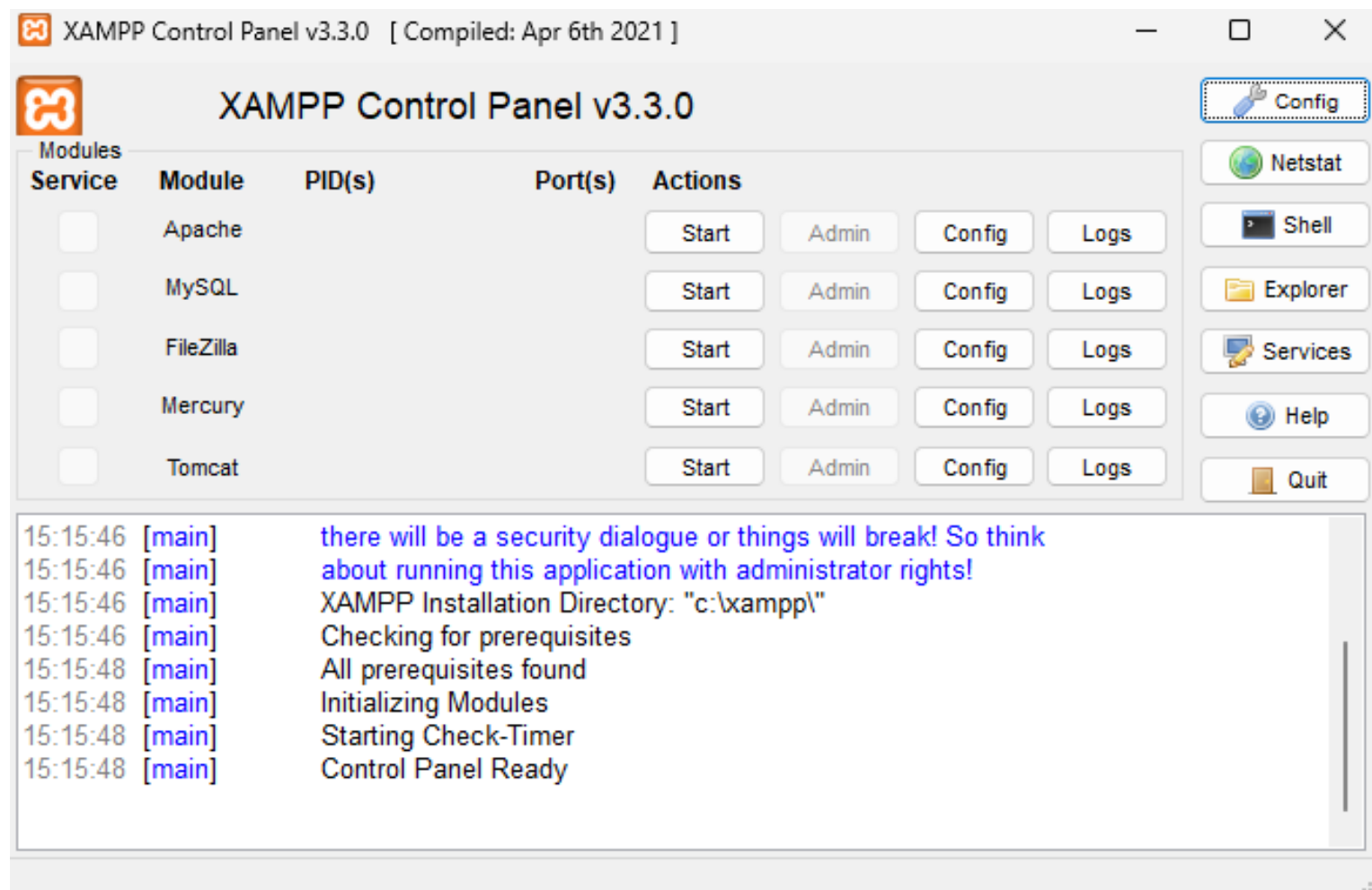
```
jdbc:mysql://host:port/database  
jdbc:oracle:thin:@host:port:database  
jdbc:db2://host:port/database
```

O formato da URL varia com o fornecedor do driver.
É preciso consultar a documentação.

Classe e método de Conexão

```
1 import java.sql.Connection;
2 import java.sql.DriverManager;
3
4 public class ConexaoBanco {
5     private static final String URL ="jdbc:mysql://localhost:3306/escola";
6     private static final String USUARIO ="root";
7     private static final String SENHA ="";
8     public static Connection conectar() {
9         try {
10             Connection conexao =DriverManager.getConnection(URL,USUARIO,SENHA);
11             System.out.println(x: "Conexão realizada!");
12             return conexao;
13         } catch (Exception erro) {
14             System.out.println(x: "Erro na conexão!");
15             System.out.println(erro.getMessage());
16             return null;
17         }
18     }
19 }
```

Xampp



Apresentar

Autenticações e Configurações

```
# The following options will be passed to all MySQL
clients
[client]
# password      = your_password
port=3306
socket="C:/xampp/mysql/mysql.sock"
```

```
<IfModule dir_module>
    DirectoryIndex index.php index.pl index.cgi
index.asp index.shtml index.html index.htm \
    default.php default.pl default.cgi
default.asp default.shtml default.html default.htm \
    home.php home.pl home.cgi home.asp
home.shtml home.html home.htm
</IfModule>
```


Consultas usando SQL em Java - DAO

- criar uma instância da classe Statement
- executar a instrução SQL

```
6 public class AlunoDAO implements OperacoesDAO {
7     private Connection conexao;
8     public AlunoDAO() {
9         conexao = ConexaoBanco.conectar();
10    }
11
12    @Override
13    public void inserir(Aluno aluno) {
14        String sql =
15            "INSERT INTO alunos(nome, idade, curso) VALUES (?, ?, ?)";
16        try {
17            PreparedStatement comando =
18                conexao.prepareStatement(sql);
19            comando.setString(parameterIndex: 1, aluno.getNome());
20            comando.setInt(parameterIndex: 2, aluno.getIdade());
21            comando.setString(parameterIndex: 3, aluno.getCurso());
22            comando.executeUpdate();
23            System.out.println(x: "Aluno cadastrado!");
24
25        } catch (Exception erro) {
26            System.out.println(x: "Erro ao inserir!");
27            System.out.println(erro.getMessage());
28        }
29    }
30 }
```

Criando Tabelas

```
Statement stmt = null;
String cmd = "CREATE TABLE usuarios (" +
            "email VARCHAR(60) NOT NULL PRIMARY KEY," +
            "nome VARCHAR(80) NOT NULL) ";

try {
    stmt = con.createStatement();
    stmt.executeUpdate(cmd);
}
catch (SQLException ex) {...}
finally {
    if (stmt != null) {
        try {
            stmt.close();
        } catch (SQLException ex) {...}
    }
}
```

ResultSet

```
1 public void consultar() {  
2     Statement s = null;  
3     Connection connection = ConexaoMySQL.getConexaoMySQL();  
4  
5     try {  
6         s = (Statement) connection.createStatement();  
7         ResultSet r = null;  
8         r = s.executeQuery("Select * from clientes");  
9  
10        System.out.println("ID    NOME");  
11        while (r.next()) {  
12            System.out.println(r.getInt("cli_codigo") + " " + r.getString("cli_nome"));  
13        }  
14  
15        r.close();  
16  
17    } catch (SQLException e) {  
18        e.printStackTrace();  
19    }  
20 }
```

ResultSet

	TINYINT	SMALLINT	INTEGER	BIGINT	REAL	FLOAT	DOUBLE	DECIMAL	NUMERIC	BIT	CHAR	VARCHAR	LONGVARCHAR	BINARY	VARBINARY	LONGVARBINARY	DATE	TIME	TIMESTAMP
getBytes	X	x	x	x	x	x	x	x	x	x	x	x	x						
getShort	x	X	x	x	x	x	x	x	x	x	x	x	x						
getInt	x	x	X	x	x	x	x	x	x	x	x	x	x						
getLong	x	x	x	X	x	x	x	x	x	x	x	x	x						
getFloat	x	x	x	x	X	x	x	x	x	x	x	x	x						
getDouble	x	x	x	x	x	X	X	x	x	x	x	x	x						
getBigDecimal	x	x	x	x	x	x	x	X	X	x	x	x	x						
getBoolean	x	x	x	x	x	x	x	x	x	X	x	x	x						
getString	x	x	x	x	x	x	x	x	x	x	X	X	x	x	x	x	x	x	x
getBytes														X	X	x			
getDate											x	x	x				X		x
getTime											x	x	x					X	x
getTimestamp											x	x	x				x	x	X
getAsciiStream											x	x	X	x	x	x			
getUnicodeStream											x	x	X	x	x	x			
getBinaryStream														x	x	X			
getObject	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x

PreparedStatement

subclasse de `java.sql.Statement`

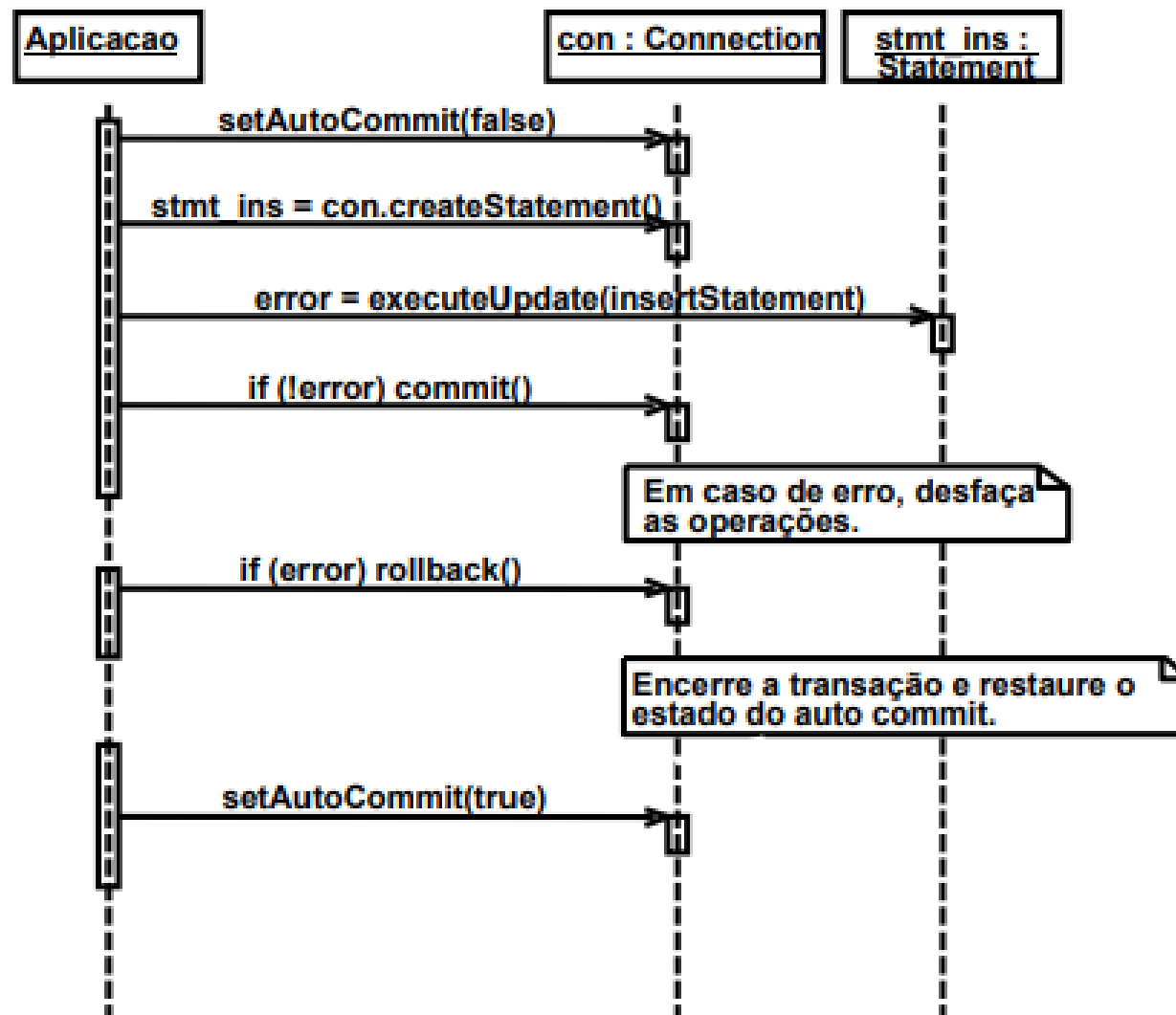
os comandos são pré-compilados

- depende do suporte do driver

os comandos SQL podem possuir um ou mais parâmetros IN

- parâmetros sem valor especificado em tempo de compilação do comando SQL
- representados por “?” (parameter marker)
- o valor precisa ser especificado antes da execução do comando

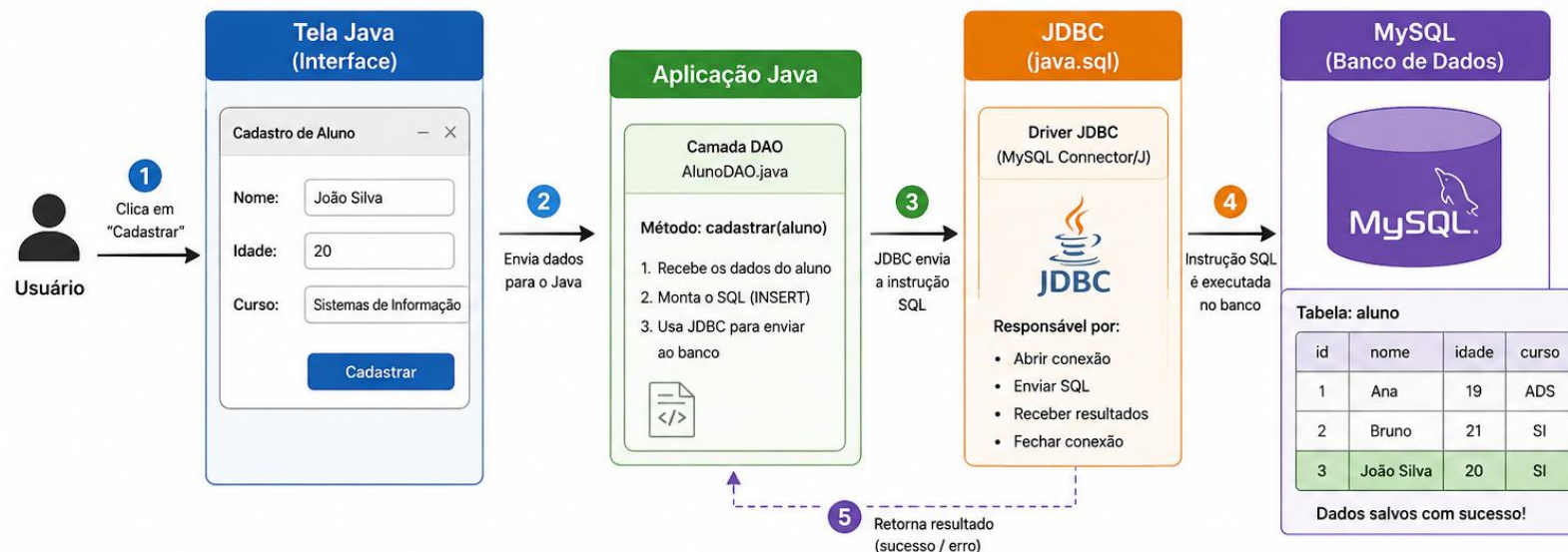
Transações



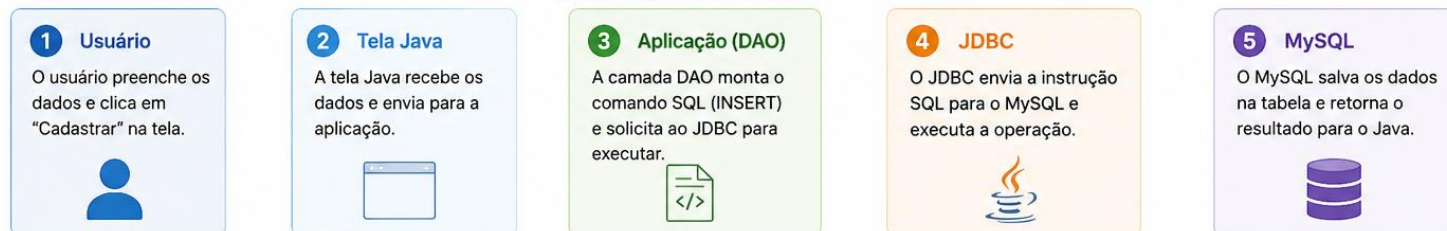
Fluxos Completos da aplicação

Tela Java → JDBC → MySQL

Fluxo completo: Cadastrar Aluno



Passo a passo do fluxo



Exemplo de SQL gerado:

```
INSERT INTO aluno (nome, idade, curso) VALUES ('João Silva', 20, 'Sistemas de Informação');
```

✓ Dados salvos!

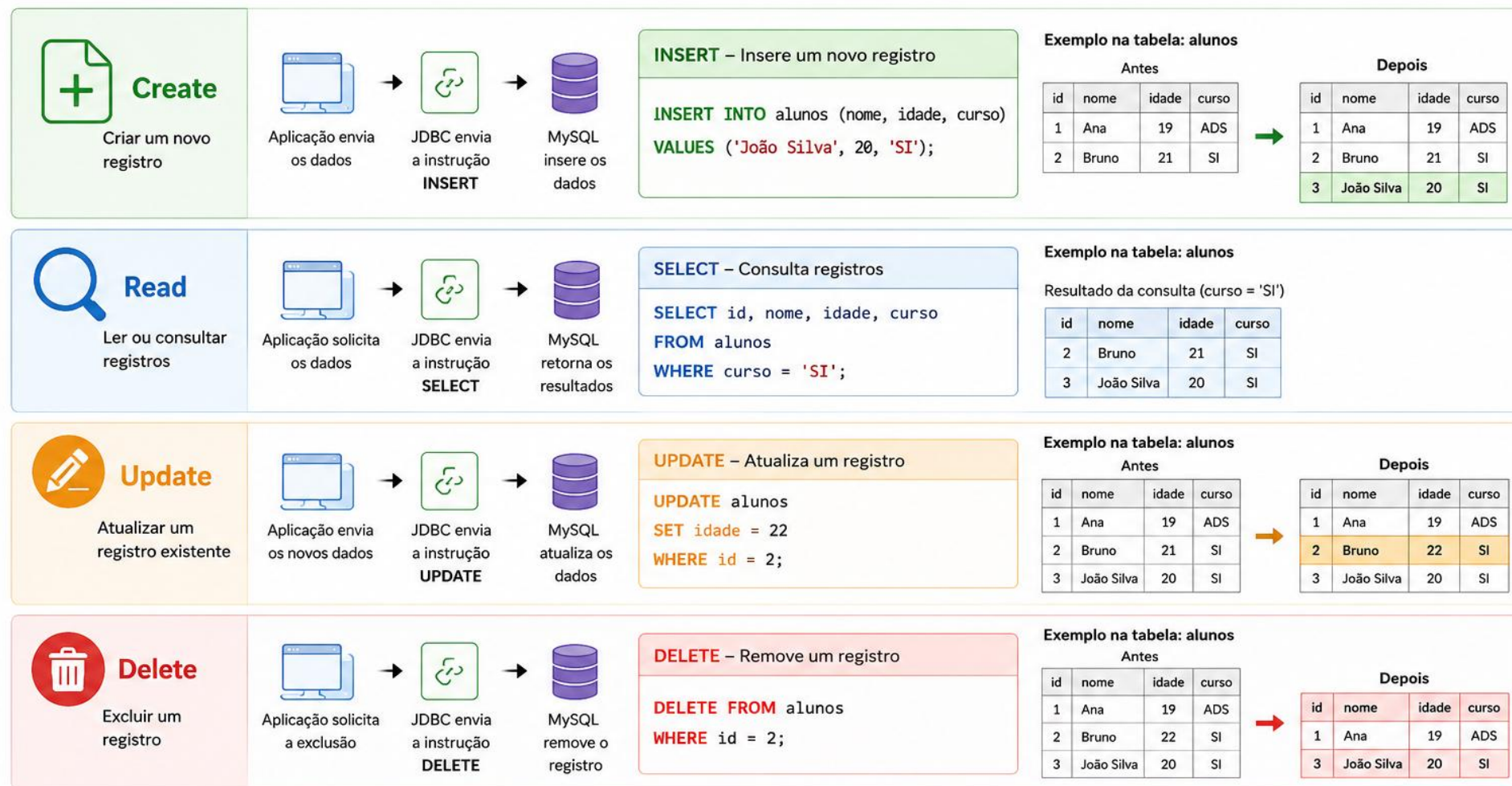
CRUD

CRUD com SQL

4 operações básicas para manipular dados em qualquer sistema



- Create → INSERT
- Read → SELECT
- Update → UPDATE
- Delete → DELETE

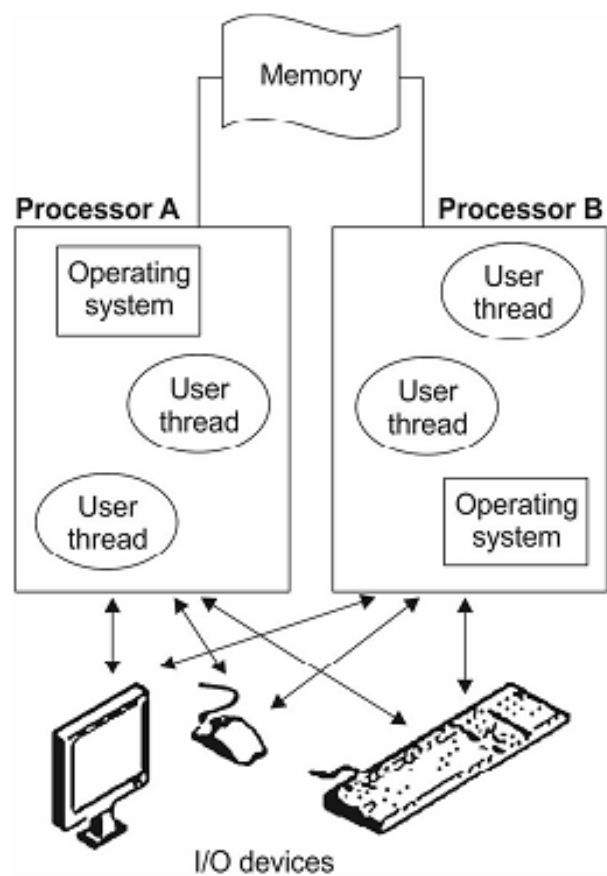


Resumo rápido

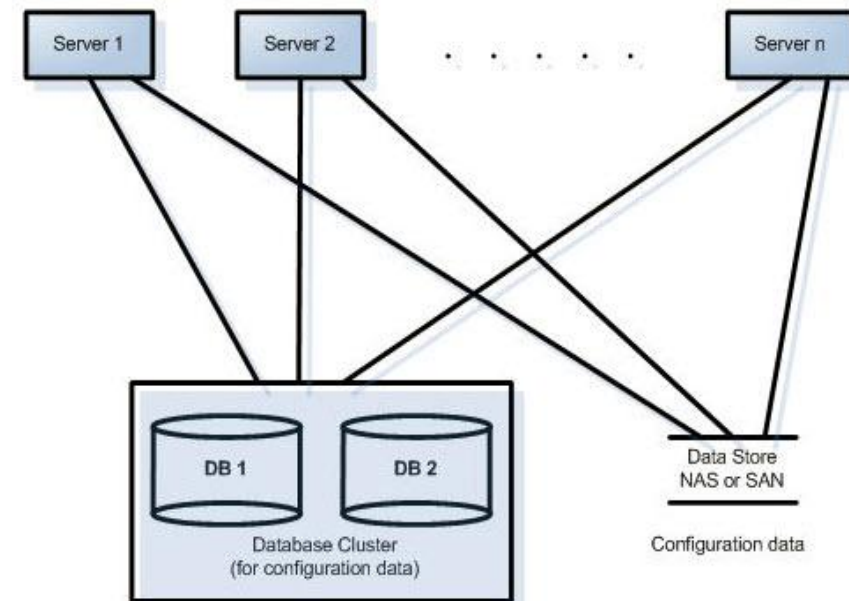
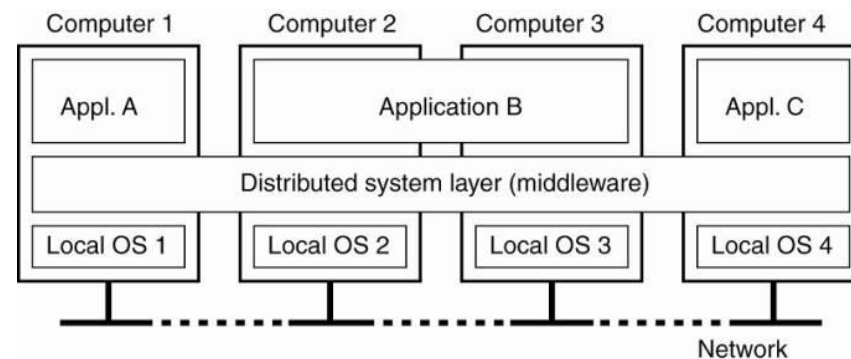
Create → INSERT → cria um novo registro | Read → SELECT → lê os registros | Update → UPDATE → altera registros | Delete → DELETE → remove registros

☆ Essas 4 operações (CRUD) estão presentes em praticamente todos os sistemas que usam banco de dados!

Arquitetura



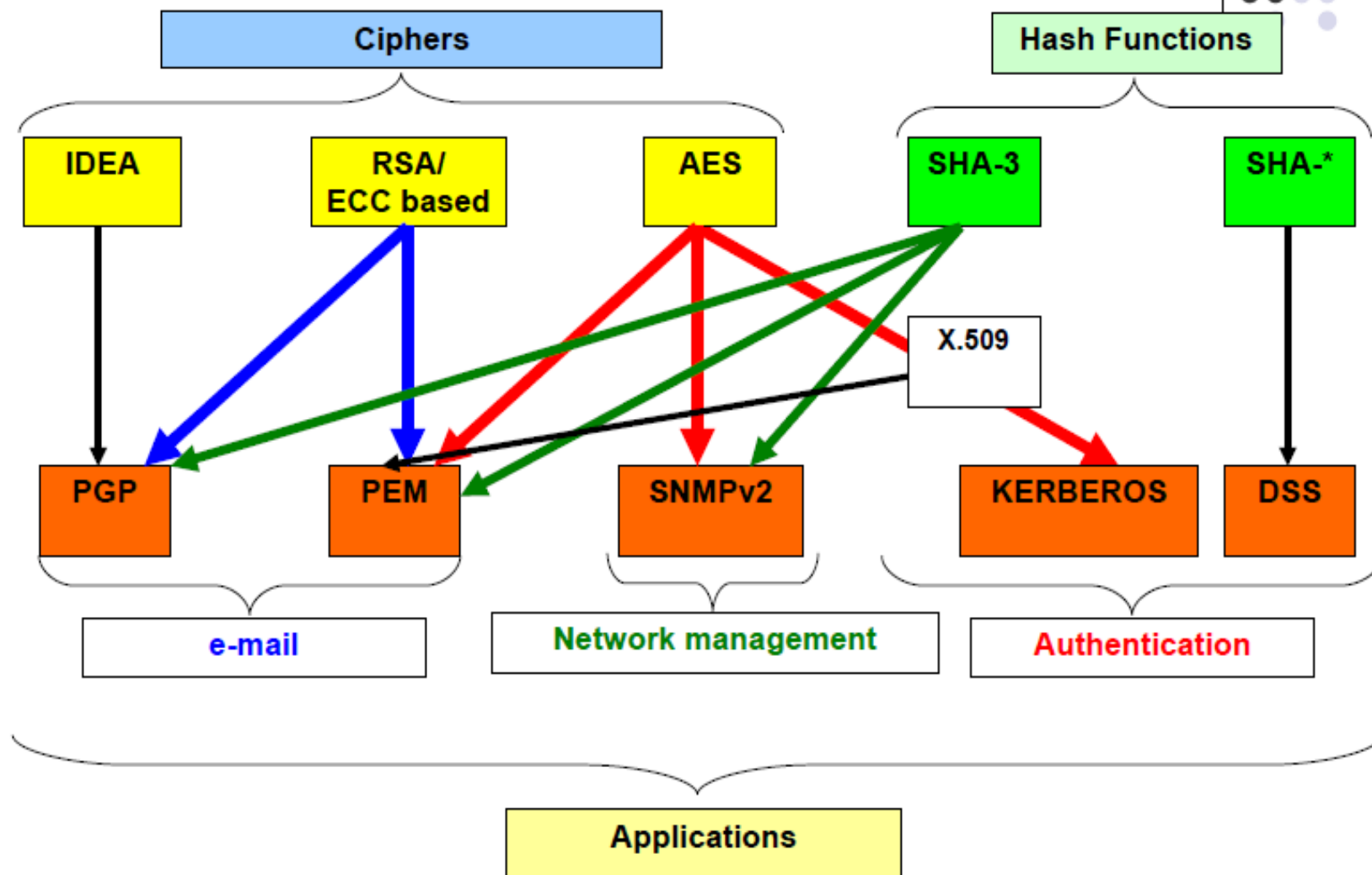
I



Segurança



Relationship: Ciphers and Applications



Segurança

“Segurança na programação não é apenas impedir invasões, mas também proteger dados, validar informações e evitar erros humanos.”

“Hoje qualquer usuário consegue cadastrar, alterar e excluir alunos. Em sistemas reais, normalmente existem níveis de acesso, como administrador, professor e aluno.”

“O tratamento de exceções melhora a segurança e estabilidade do sistema, impedindo falhas inesperadas e ajudando na identificação de problemas.”

Segurança

2020

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	Instantly	Instantly	Instantly
6	Instantly	Instantly	Instantly	1 sec	5 secs
7	Instantly	Instantly	25 secs	1 min	6 mins
8	Instantly	5 secs	22 mins	1 hour	8 hours
9	Instantly	2 mins	19 hours	3 days	3 weeks
10	Instantly	58 mins	1 month	7 months	5 years
11	2 secs	1 day	5 years	41 years	400 years
12	25 secs	3 weeks	300 years	2k years	34k years
13	4 mins	1 year	16k years	100k years	2m years
14	41 mins	51 years	800k years	9m years	200m years
15	6 hours	1k years	43m years	600m years	15bn years
16	2 days	34k years	2bn years	37bn years	1tn years
17	4 weeks	800k years	100bn years	2tn years	93tn years
18	9 months	23m years	61tm years	100tn years	7qd years

Segurança

TIME IT TAKES A HACKER TO BRUTE FORCE YOUR PASSWORD IN 2022

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	Instantly	Instantly	Instantly
6	Instantly	Instantly	Instantly	Instantly	Instantly
7	Instantly	Instantly	2 secs	7 secs	31 secs
8	Instantly	Instantly	2 mins	7 mins	39 mins
9	Instantly	10 secs	1 hour	7 hours	2 days
10	Instantly	4 mins	3 days	3 weeks	5 months
11	Instantly	2 hours	5 months	3 years	34 years
12	2 secs	2 days	24 years	200 years	3k years
13	19 secs	2 months	1k years	12k years	202k years
14	3 mins	4 years	64k years	750k years	16m years
15	32 mins	100 years	3m years	46m years	1bn years
16	5 hours	3k years	173m years	3bn years	92bn years
17	2 days	69k years	9bn years	179bn years	7tn years
18	3 weeks	2m years	467bn years	11tn years	438tn years



> Learn about our methodology at hivesystems.io/password

Segurança

TIME IT TAKES A HACKER TO BRUTE FORCE YOUR PASSWORD IN 2023

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	Instantly	Instantly	Instantly
6	Instantly	Instantly	Instantly	Instantly	Instantly
7	Instantly	Instantly	1 sec	2 secs	4 secs
8	Instantly	Instantly	28 secs	2 mins	5 mins
9	Instantly	3 secs	24 mins	2 hours	6 hours
10	Instantly	1 min	21 hours	5 days	2 weeks
11	Instantly	32 mins	1 month	10 months	3 years
12	1 sec	14 hours	6 years	53 years	226 years
13	5 secs	2 weeks	332 years	3k years	15k years
14	52 secs	1 year	17k years	202k years	1m years
15	9 mins	27 years	898k years	12m years	77m years
16	1 hour	713 years	46m years	779m years	5bn years
17	14 hours	18k years	2bn years	48bn years	380bn years
18	6 days	481k years	126bn years	2tn years	26tn years



› Learn how we made this table at hivesystems.io/password

Segurança

TIME IT TAKES A HACKER TO BRUTE FORCE YOUR PASSWORD IN 2024

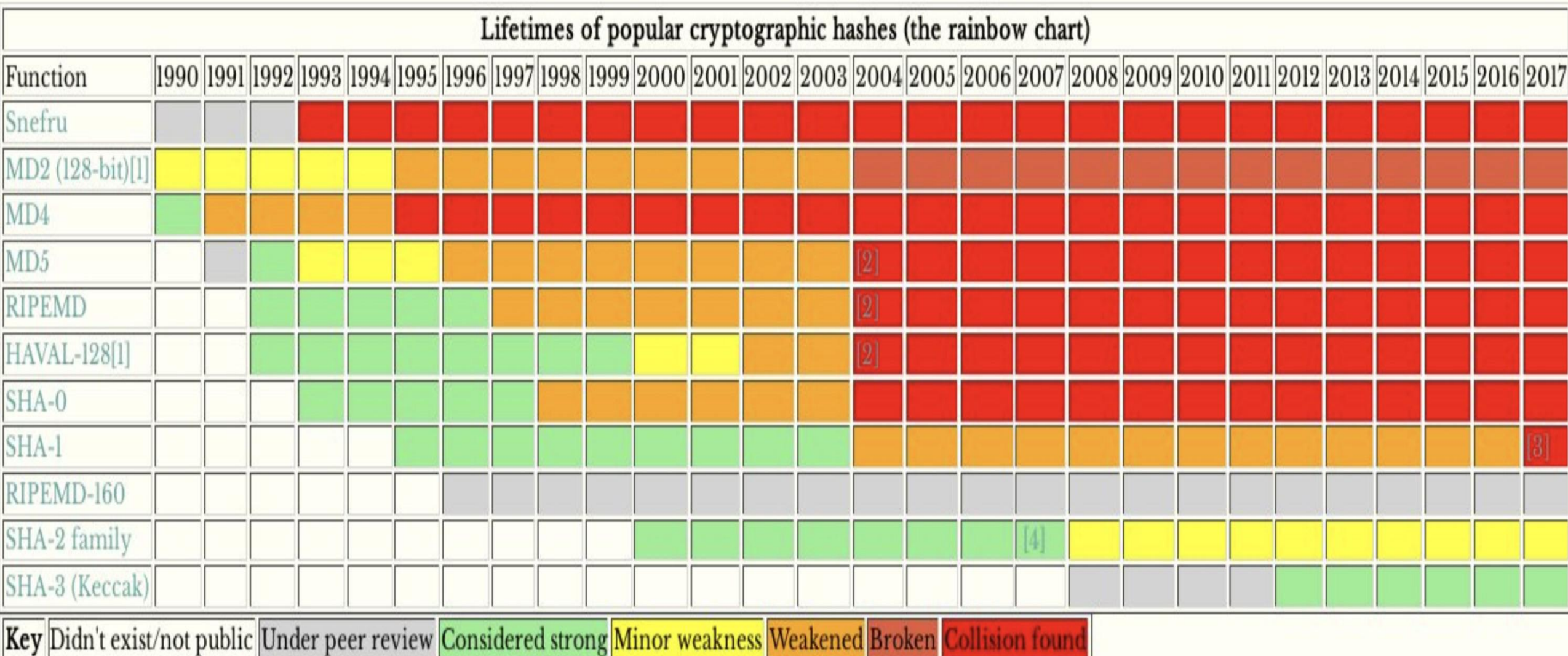
Hardware: 12 x RTX 4090 | Password hash: bcrypt

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	3 secs	6 secs	9 secs
5	Instantly	4 secs	2 mins	6 mins	10 mins
6	Instantly	2 mins	2 hours	6 hours	12 hours
7	4 secs	50 mins	4 days	2 weeks	1 month
8	37 secs	22 hours	8 months	3 years	7 years
9	6 mins	3 weeks	33 years	161 years	479 years
10	1 hour	2 years	1k years	9k years	33k years
11	10 hours	44 years	89k years	618k years	2m years
12	4 days	1k years	4m years	38m years	164m years
13	1 month	29k years	241m years	2bn years	11bn years
14	1 year	766k years	12bn years	147bn years	805bn years
15	12 years	19m years	652bn years	9tn years	56tn years
16	119 years	517m years	33tn years	566tn years	3qd years
17	1k years	13bn years	1qd years	35qd years	276qd years
18	11k years	350bn years	91qd years	2qn years	19qn years



› Learn more about this at hivesystems.com/password

Cryptographic hashes



Interceptações

The image shows a Wireshark packet capture of an HTTP POST request. The filter bar at the top is set to `http.request.method == POST`. The packet list shows packet 2910, which is an HTTP POST request to `/userinfo.php` from source IP 192.168.15.9 to destination IP 44.228.249.3. The packet details pane on the left shows the structure of the request, including the Host, User-Agent, Accept, Accept-Language, Accept-Encoding, Content-Type, and various headers. The packet bytes pane on the right shows the raw data of the request, including the HTTP headers and the body of the request, which is URL-encoded form data. The status bar at the bottom indicates that 4338 packets are displayed, with 1 packet shown and 0 dropped.

Ethernet: en0

Filter: `http.request.method == POST`

No.	Time	Source	Destination	Protocol	Length	Info
2910	16.664575	192.168.15.9	44.228.249.3	HTTP	623	POST /userinfo.php HTTP/1.1 (application/x-www-form-urlencoded)

Frame 2910: 623 bytes on wire (4984 bits), 623 bytes captured (4984 bits) on interface 0

Ethernet II, Src: Apple_30:82:4c (a8:60:b6:30:82:4c), Dst: Tellesco_08:00:27:00:00:00

Internet Protocol Version 4, Src: 192.168.15.9, Dst: 44.228.249.3

Transmission Control Protocol, Src Port: 59786, Dst Port: 80, Seq: 1664575, Win: 0, Len: 623

Hypertext Transfer Protocol

POST /userinfo.php HTTP/1.1\r\nHost: testphp.vulnweb.com\r\nUser-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/112.0\r\nAccept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8\r\nAccept-Language: en-US,en;q=0.5\r\nAccept-Encoding: gzip, deflate\r\nContent-Type: application/x-www-form-urlencoded\r\nContent-Length: 35\r\nOrigin: http://testphp.vulnweb.com\r\nConnection: keep-alive\r\nReferer: http://testphp.vulnweb.com/login.php\r\nUpgrade-Insecure-Requests: 1\r\n\r\n[Full request URI: http://testphp.vulnweb.com/userinfo.php]\r\n[HTTP request 1/3]\r\n[Response in frame: 2951]\r\n[Next request in frame: 2953]\r\nFile Data: 35 bytes

HTML Form URL Encoded: application/x-www-form-urlencoded

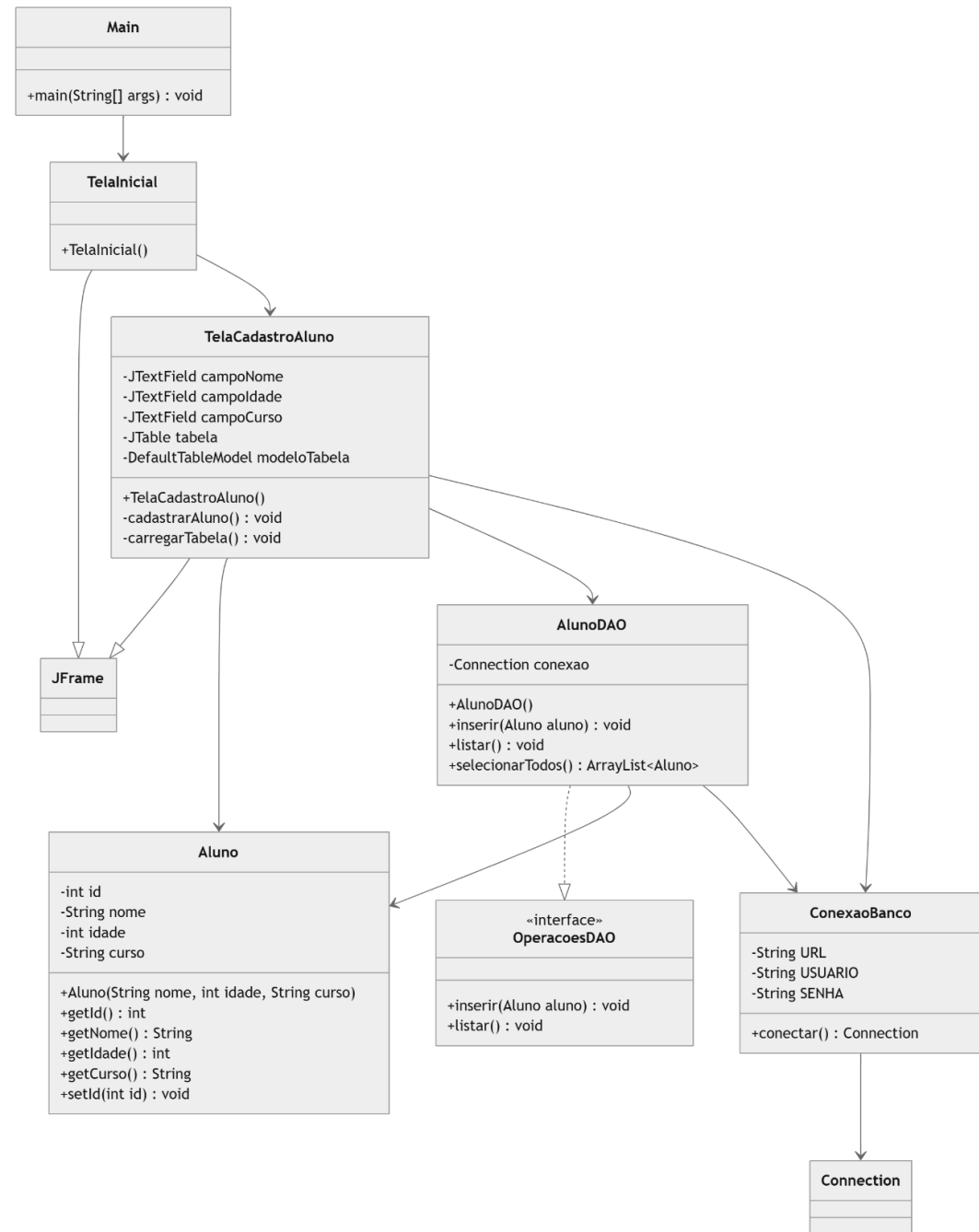
Form item: "uname" = "charles.miers"

Form item: "pass" = "test123456"

Text item (text), 20 bytes

Packets: 4338 · Displayed: 1 (0.0%) · Dropped: 0 (0.0%) · Profile: Default

Exemplo



Atividades - Perguntas

1. Quais bancos de dados além do MySQL podem ser utilizados com Java? Pesquise exemplos utilizados no mercado e suas principais características.
2. Como funciona a conexão do Java com bancos de dados como PostgreSQL, Oracle Database e MongoDB? Existe diferença nos drivers JDBC utilizados?
3. Quais empresas ou sistemas famosos utilizam Java integrado a bancos de dados? Pesquise exemplos reais do mercado.
4. Qual a diferença entre bancos relacionais (SQL) e não relacionais (NoSQL) quando utilizados em aplicações Java?
5. Pesquise quais vantagens e desvantagens existem ao utilizar Java com diferentes bancos de dados no desenvolvimento de sistemas modernos.

Atividades - Perguntas

1. O que é SQL Injection e como ele pode comprometer um sistema?
2. Por que o uso de PreparedStatement é mais seguro do que concatenar SQL com texto?
3. Quais riscos podem ocorrer ao armazenar senhas diretamente no banco de dados?
4. O que pode acontecer se um sistema Java não validar os dados digitados pelo usuário?
5. Qual a importância do login e controle de acesso em sistemas com banco de dados?
6. Por que não é recomendado utilizar o usuário root do banco em aplicações?
7. O que pode acontecer se uma conexão com banco de dados ficar aberta sem necessidade?
8. Quais informações pessoais precisam de maior proteção em sistemas digitais?
9. Como a criptografia pode ajudar na segurança de aplicações Java com banco de dados?

Atividades

Análise do código

Atividades

Melhorar a qualidade do sistema.

- idade negativa;
- idade maior que 120;
- nome vazio;
- nome com números;
- curso vazio.

Permitir ordenar alunos.

- Ordenar por Nome
- Ordenar por Idade

Implementar pesquisa de alunos por nome.

- possuir um campo de pesquisa;
- possuir botão Buscar;
- mostrar apenas os alunos encontrados.

Estatística do Sistema - BI

quantidade total de alunos;

média de idade;

aluno mais velho;

aluno mais novo.

Tentar implementar um gráfico ou uma visualização

Referência

INSTITUTO METRÓPOLE DIGITAL (IMD). Banco de Dados: Aula 19 - Utilizando SQL em Java. Natal: Universidade Federal do Rio Grande do Norte, [s.d.]. Disponível em: Material didático IMD/UFRN. Acesso em: 13 maio 2026.

Materiais adaptados professor Dr Charles Christian Miers – UDESC – Joinville.

This work is licensed under a Creative Commons license



Attribution-ShareAlike 4.0

<http://creativecommons.org/licenses/by-sa/4.0/>



PELO FUTURO DO TRABALHO

sesisc.org.br     **sc.senai.br**

0800 048 1212

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC